

LC

مقارنة التطوير المعماري في هندسة البرمجيات والتشفير المتقدم

مشروع تطبيق الاتصال المحلي المباشر (LOCAL CONTACT P2P)

وثيقة هندسية وأمنية تبرز التحول النوعي للتطبيق من نموذج أولي يعتمد على مفتاح تشفير ثابت إلى نظام شبكي P2P مهيكل وفق أحدث معايير هندسة البرمجيات (SRS, Baseline, RTM) ومحسن بمنظومة تشفير ديناميكية (AES-256-GCM, ECDH P-256, HKDF-SHA256, Android Keystore, TOFU).

لجنة المناقشة والتحكيم:

أستاذ هندسة البرمجيات + أستاذ التشفير

إعداد الطالب المنفذ:

مشعل

نتائج الاختبارات الآلية (Unit Tests):

33 ناجح من أصل 33 (100% JVM)

حالة بناء الحزمة (Build Status):

BUILD SUCCESSFUL (APK جاهز)

1 الملخص التنفيذي وأهداف المشروع

مشروع **LOCAL CONTACT** هو منصة اتصال مباشر بين أجهزة Android عبر الشبكة المحلية (Wi-Fi LAN أو Hotspot) دون الاعتماد على خادم مركزي أو إنترنت. قبل بدء التطوير، كان التطبيق يمتلك وظائف تشغيلية ولكن كان يفتقر إلى التوثيق المعماري الرسمي ويعتمد على **مفتاح تشفير ثابت** مشتق من عبارة مرور داخل الكود، مما يمثل ثغرة معمارية لكافة الجلسات المسجلة.

تم تنفيذ التطوير عبر مسارين متوازيين: **(1) هندسة البرمجيات:** توثيق المتطلبات (SRS)، تثبيت خط الأساس (Baseline)، بناء مصفوفة التتبع (RTM)، وفصل إدارة المفاتيح بنمط **(2) Provider. التشفير وأمن الاتصال:** الحفاظ على خوارزمية AES-256-GCM، وإضافة تبادل المفاتيح المنحني P-ECDH مع دالة الاشتقاق HKDF-SHA256، وحفظ هوية الجهاز داخل معالج Android Keystore، وتثبيت الهوية بنموذج TOFU والتوقيع الحتمي.

2 خريطة الملفات والمجلدات: ما تم إضافته وتعديله ولماذا؟

يوضح الجدول التالي كافة الملفات المضافة والمعدلة في المشروع والهدف الهندسي والأمني الدقيق منها:

المسار والملف	نوع الإجراء	المسؤولية	لماذا تم الإجراء؟ (القيمة المضافة في المشروع)
docs/SRS.md	ملف جديد	مواصفات المتطلبات	توثيق 31 مطلباً وظيفياً (FR-01 إلى FR-31) لربط الكود بمراجعة رسمية.
docs/Baseline.md	ملف جديد	خط الأساس المعماري	تحديد الحالة المرجعية للنظام قبل وبعد التحسينات لتسهيل الصيانة.
docs/TraceabilityMatrix.md	ملف جديد	مصفوفة تتبع RTM	ربط صارم: <i>مطلب</i> – <i>ملف الكود</i> – <i>كلاس الاختبار</i> – <i>حالة الإنجاز</i> .
.../network/KeyProvider.kt	كود جديد	واجهة تجريد المفاتيح	فصل إدارة المفاتيح عن محرك التشفير بنمط Provider لتطبيق مبدأ DIP.
.../network/EcdhEngine.kt	كود جديد	تبادل المفاتيح المنحني	توليد مفاتيح P-256 واشتقاق مفاتيح جلسة ديناميكية عبر HKDF-SHA256.
.../network/KeystoreIdentityManager.kt	كود جديد	هوية الجهاز بالعتاد	إنشاء زوج مفاتيح EC غير قابل للتصدير داخل Android Keystore للتوقيع.
.../network/HandshakeCryptoUtils.kt	كود جديد	التوقيع المرتبط بالسياق	بناء مصفوفة بايت حتمية تضم سياق الجلسة لمنع هجمات إعادة البث.
.../network/PeerTrustStore.kt	كود جديد	مخزن ثقة الأقران	تخزين بصمات SHA-256 لمفاتيح الأجهزة بنموذج TOFU واكتشاف تغيرها.
.../network/PeerSessionKeyManager.kt	كود جديد	إدارة دورة حياة الجلسة	حفظ مفاتيح الجلسات وإدارة حالات الأمان ومنع هجمات Downgrade.
.../network/LocalCryptoEngine.kt	تعديل هيكلي	محرك التشفير المتماثل	الاعتماد على KeyProvider والحفاظ الكامل على خوارزمية AES-256-GCM.
.../network/LocalP2PEngine.kt	تعديل ودمج	الاتصال والمصافحة	دمج رسائل المصافحة الثلاثية وربط الدردشة المباشرة بمفاتيح الجلسات.
app/src/test/.../	اختبارات جديدة	حزم اختبارات JVM	4 ملفات اختبارات للمحرك، التبادل، الجلسات، والمصافحة (33 اختباراً ناجحاً).

3 الجزء الأول: منهجية هندسة البرمجيات وإدارة الجودة



1.3 إنجازات هندسة البرمجيات المحققة:

- ✓ **SRS.md**: وثيقة رسمية للمتطلبات تشمل 31 مطلباً (FR-01 إلى FR-31) تغطي الاستكشاف، المراسلة، نقل الملفات، المكالمات، الأمان، وقاعدة البيانات.
- ✓ **Baseline.md**: خط أساس معماري يوثق حالة النظام الأصلية، منافذ السوكيت (8888-8891)، تدفق الحزم، والفجوات المعروفة قبل التحسين.
- ✓ **TraceabilityMatrix.md**: مصفوفة تتبع صارمة تمنع الخلط بين ما هو مختبر على JVM، وما هو مدمج في الكود، وما هو قيد التحسين المستقبلي.
- ✓ **KeyProvider Pattern**: تجريد إدارة المفاتيح لعزل مسؤولية اشتقاق المفاتيح عن محرك التشفير وتسهيل دعم مفاتيح متعددة لاحقاً.

2.3 جدول نتائج الاختبارات الآلية المنفذة (33 / 33 اختباراً ناجحاً):

كلاس الاختبار	ما يتم التحقق منه واختباره برمجياً	العدد	النتيجة
LocalCryptoEngineTest	صحة AES-256-GCM، رفض المفتاح الخاطئ، كشف التلاعب بالنص المشفر وبالوسم (Tag)، اشتقاق Nonce الكتل، ودعم UTF-8 ونصوص JSON.	10 فحوصات	10/10 ناجح
EcdhEngineTest	توليد أزواج P-256 المؤقتة، إثبات اتفاق طرفين مستقلين على نفس السر المشترك، واشتقاق مفتاح جلسة متطابق عبر HKDF-SHA256.	8 فحوصات	8/8 ناجح
PeerSessionKeyManagerTest	إدارة جلسات الأقران، التحقق من صلاحية الجلسات النشطة، انتهاء صلاحية المصافحة بعد 30 ثانية، واسترجاع المفتاح الصحيح لكل جهاز.	4 فحوصات	4/4 ناجح
HandshakeHardeningTest	التحقق من التوقيع المرتبط بالسياق، كشف التلاعب في معرف الجلسة أو الوقت، وحفظ ومطابقة بصمة الهوية بمخزن ثقة TOFU.	8 فحوصات	8/8 ناجح
اختبارات تراكيب البيانات	فحوصات سلامة نماذج البيانات ونظام الكبسولات في المشروع.	3 فحوصات	3/3 ناجح
المجموع الكلي للاختبارات المنفذة والمحققة بنجاح تام:		33 فحصاً	33 / 33 (100%)

دلالة نتيجة 33/33 الهندسية:

تثبت هذه النتيجة السلامة الخوارزمية للمحرك البرمجي وإدارة الجلسات على بيئة JVM، لكنها لا تعني اختبار الاتصال الميداني بين هاتفين حقيقيين عبر راوتر Wi-Fi.

4 الجزء الثاني: التشفير المتقدم وأمن الاتصال الشبكي

كان النظام القديم يعتمد على مفتاح AES-256 ثابت مشتق عبر PBKDF2 من عبارة مرور موحدة داخل الكود. تم تطوير النظام ليعتمد على منظومة تشفير حديثة:

⚡ تبادل المفاتيح: ECDH (NIST P-256)

توليد مفاتيح مؤقتة وحساب السر المشترك رياضياً على المنحني `secp256r1` محققاً سرية التوجيه المستقبلي (Forward Secrecy).

🔒 التشفير المتماثل: AES-256-GCM

مفتاح 256 بت مع وسم مصادقة 128 بت ورقم عشوائي Nonce بطول 12 بايت، يضمن السرية وسلامة البيانات ويرفض أي تلاعب بالحزم.

🔑 هوية العتاد: Android Keystore

حفظ مفاتيح الهوية غير المتماثل في العتاد الآمن (TEE) وتوقيع بيانات المصادقة بـ ECDSA دون إمكانية تصدير المفتاح الخاص.

🔗 اشتقاق المفاتيح: HKDF-SHA256

تطبيق معيار RFC 5869 في مرحلتي Extract و Expand لاستنتاج مفاتيح جلسة عشوائية متماثلة ومحمية بسياق الجلسة.

1.4 حماية المصادقة: التوقيع المرتبط بالسياق (Context-Bound) ونموذج TOFU

🔒 تثبيت الهوية (Trust On First Use - TOFU)

عند أول اتصال يُحفظ بصمة مفتاح القرين (SHA-256 Fingerprint) بمخزن الثقة. في الاتصالات اللاحقة، يُرفض الاتصال إذا تغير المفتاح، مما يحمي الأجهزة من انتحال الهوية بعد المصادقة الأولى.

🔒 التوقيع المرتبط بالسياق (Handshake Hardening)

يتم تجميع حقول المصادقة في هيكل باينري حتمي يضم: (إصدار البروتوكول 2، معرف المرسل، المستقبل، الجلسة، الوقت، والمفتاح المؤقت). أي تعديل في سياق الجلسة يؤدي فوراً لفشل التحقق بالتوقيع الرقمي.

2.4 الشفافية الأكاديمية: مصفوفة نطاق المفاتيح الفعلي (Key Scope Matrix)

يوضح الجدول التالي ما تم تحويله فعلياً لمفاتيح الجلسات المشتقة، وما لا يزال يعمل بالمفتاح الشبكي القديم:

المسار / نوع البيانات	المفتاح المستخدم حالياً	المنفذ والبروتوكول	حالة الحماية والأمان
استكشاف الأجهزة (Discovery)	المفتاح الشبكي: <code>networkKey</code>	8888 (UDP Broadcast)	طبيعي (مطلوب عاماً لاكتشاف الأولي للأجهزة).
الدردشة المباشرة (Direct Chat)	مفتاح الجلسة: <code>sessionKey</code>	8888 (UDP Unicast)	محمي بـ جلسة ECDH ديناميكية لكل قرين
غرف المحادثة الجماعية (Rooms)	المفتاح الشبكي: <code>networkKey</code>	8888 (UDP Multicast)	مسار قديم للتوافق (يحتاج Group Key مستقبلاً).
نقل وتبادل الملفات (Files)	المفتاح الشبكي: <code>networkKey</code>	8891 (TCP Stream)	مشفر بـ 64KB بمفتاح الشبكة القديم.
المكالمات الصوتية (Audio)	المفتاح الشبكي: <code>networkKey</code>	8889 (UDP Stream)	مشفر بحزم PCM بمفتاح الشبكة القديم.
مكالمات الفيديو والشاشة	المفتاح الشبكي: <code>networkKey</code>	8890 (UDP Stream)	مشفر بإطارات JPEG بمفتاح الشبكة القديم.

5 المقارنات الشاملة (قبل وبعد) والأمانة الأكاديمية

1.5 مقارنة هندسة البرمجيات والتشفير:

المجال	قبل التحسين (الحالة السابقة)	بعد التحسين (النسخة المنجزة)
توثيق المتطلبات	فهم شفوي وضملي داخل الكود	موثقة في docs/SRS.md لـ 31 متطلباً وظيفياً ومعايير قبول.
الخط المرجعي المعماري	غير موحد ويصعب تمييز الإضافات	موثق في docs/Baseline.md كنقطة ارتكاز معمارية واضحة.
تتبع المتطلبات RTM	غير موجود	مسار هندسي موثق في docs/TraceabilityMatrix.md .
تجريد إدارة المفاتيح	مقترن بشدة داخل محرك التشفير	معزول ومجرد عبر واجهة KeyProvider وفق مبادئ SOLID.
مصدر مفتاح التشفير	عبارة مرور ثابتة مشتقة عبر PBKDF2	تبادل ديناميكي ECDH P-256 واشتقاق HKDF-SHA256 لكل جلسة.
سرية التوجيه المستقبلي	منعدمة تماماً	محققة في محادثات الجلسات (المفاتيح المؤقتة تُتلف فوراً).
هوية الجهاز المشفرة	غير موجودة (معرف عشوائي فقط)	مفتاح EC غير قابل للتصدير داخل Android Keystore بتوقيع ECDSA.
حماية المصافحة من العبث	غير موجودة	توقيع رقمي موحد مرتبط بالسياق (Context-Bound Signature).
التحقق من هوية الأقران	قبول أي جهاز على الشبكة	نموذج Trust On First Use (TOFU) ومطابقة بصمات SHA-256.

2.5 حدود التحقق والأمانة الأكاديمية (ما تبقى للتحسين المستقبلي):

حدود العمل الحالي والمسارات المفتوحة للتحسين:

- ⚠️ **التحقق اليدوي (SAS (Short Authentication String):** حماية MITM في الاتصال الأول تظل "جزئية" لغياب مقارنة كود صوتي أو مسح QR بين المستخدمين في أول لقاء.
- ⚠️ **الحماية الكاملة من إعادة الإرسال (Replay Protection):** التوقيع يغطي سياق المصافحة فقط، ولا توجد حتى الآن أرقام تسلسلية للحزم الشبكية (Sequence Numbers) مع نافذة رفض للمكرر.
- ⚠️ **فحص سلامة الملف كاملاً (Whole-File SHA-256):** يتم التحقق من سلامة كل كتلة 64KB عبر AES-GCM Tag، ولكن لا يوجد فحص بصمة شامل للملف الكامل بعد تجميعه.
- ⚠️ **مفاتيح المجموعات (Group Keys):** غرف المحادثة الجماعية ما زالت تستخدم المفتاح الشبكي القديم المشترك.
- ⚠️ **الاختبار الميداني المباشر:** نجاح 33/33 اختباراً تم على بيئة JVM ولم يُختبر ميدانياً بين هاتفين حقيقيين عبر راوتر فيزيائي.

6 دليل المناقشة الشفوي والأسئلة المتوقعة من الدكاترة

أولاً: أسئلة أستاذ هندسة البرمجيات (Software Engineering):

هندسة متطلبات

س1: ما الجدوى الهندسية لتوثيق SRS و Baseline لمشروع كان يعمل برمجياً بالفعل؟

الإجابة: البرمجيات الاحترافية لا تقاس بمجرد تشغيل الكود، بل بـ **قابلية الصيانة (Maintainability)** والتتبع وإثبات مطابقة الوظائف لمعايير القبول. بدون SRS لا نملك توصيفاً رسمياً للحدود التشغيلية أو الحالات الحدية، وبدون Baseline لا يمكن لأي مطور قياس أثر التعديلات المعمارية.

المرجع: docs/SRS.md و docs/Baseline.md

تصميم معماري

س2: أين يظهر تطبيق مبادئ SOLID ونمط التصميم (Design Patterns) في الكود؟

الإجابة: طبقنا مبدأ DIP (**عكس التبعية**) ونمط **Strategy/Provider Pattern** عبر واجهة **KeyProvider**. كان محرك التشفير مقترناً بشدة بدوال اشتقاق المفتاح الثابت، فقمنا بفصل المسؤوليات بحيث يعتمد التشفير على تجريد مجرد، مما أتاح إضافة مفاتيح الجلسات دون المساس بخوارزميات التشفير AES-GCM.

المرجع: app/src/main/java/com/example/network/KeyProvider.kt

ثانياً: أسئلة أستاذ التشفير وأمن المعلومات (Cryptography):

تشفير متماثل

س1: لماذا تم اختيار خوارزمية AES-GCM بالتحديد؟ وما وسم المصادقة؟

الإجابة: لأن AES-GCM يوفر **Authenticated Encryption (AEAD)** في خطوة واحدة، حيث يدمج السرية مع سلامة البيانات. يولد النمط وسمّاً بطول 128 بت يسمى **Authentication Tag** يضمن كشف أي تعديل في النص المشفر أو الـ Nonce، ويرفض فك التشفير تلقائياً عند التلاعب، محققاً حماية تامة من هجمات Bit-Flipping.

المرجع: LocalCryptoEngine.kt (encrypt / decrypt)

تبادل مفاتيح

س2: كيف يحقق بروتوكول ECDH سرية التوجيه المستقبلي (Forward Secrecy)؟

الإجابة: نستخدم منحى NIST P-256 لتوليد أزواج مفاتيح مؤقتة (**Ephemeral**) لكل جلسة اتصال مستقلة. الطرفان يحسبان السر المشترك ثم يُتلف المفتاح الخاص المؤقت فوراً من الذاكرة. في حال اختراق الجهاز لاحقاً، يستحيل رياضياً فك تشفير حزم الجلسات السابقة المسجلة. ودور HKDF-SHA256 هو تنقية السر المشترك وتحويله لمفتاح AES متماثل نقي عشوائياً.

المرجع: EcdhEngine.kt (deriveSharedSecret / deriveSessionKey)

تحليل ثغرات

س3: لماذا وصفتم حماية رجل المنتصف (MITM) بأنها "جزئية وليست كاملة"؟

الإجابة: دقة علمية: لأننا نعتمد نموذج **TOFU** لغياب خادم شهادات مركزي (No CA). في أول اتصال، إذا تواجد مهاجم نشط (Active MITM) يمكنه استبدال المفتاح العام. نموذج TOFU يحمي بامتياز بدءاً من الاتصال الثاني؛ وللحماية التامة في أول لقاء يلزم قناة خارجية (SAS أو QR Code).

المرجع: PeerTrustStore.kt و PeerSessionKeyManager.kt

7 الخريطة المرجعية السريعة في الكود (دوال ومتغيرات هامة)

الميزة / الوظيفة	الملف	اسم المتغير أو الدالة في الكود
منفذ وسوكيت الاستكشاف	LocalP2PEngine.kt	discoverySocket / PORT_DISCOVERY = 8888
إرسال واستقبال الرسائل	LocalP2PEngine.kt	handleIncomingPacket() / sendMessage()
تشفير وفك تشفير AES-GCM	LocalCryptoEngine.kt	decrypt() / encrypt()
توليد واشتقاق مفاتيح ECDH	EcdhEngine.kt	deriveSessionKey() / generateEphemeralKeyPair()
هوية وتوقيع Android Keystore	KeystoreIdentityManager.kt	/ getOrCreateIdentityKeyPair() signHandshakePayload()
التوقيع المرتبط بالسياق وبصمة TOFU	HandshakeCryptoUtils.kt	/ buildCanonicalSignatureInput() computeFingerprint()
مخزن الثقة وإدارة الجلسات	PeerSessionKeyManager.kt	PeerSecurityStatus / getSessionKey()
منافذ نقل الملفات، الصوت، والفيديو	File/Audio/VideoEngine.kt	, 8889 (Audio UDP) , 8891 (TCP File) 8890 (Video UDP)

8 الصياغة النموذجية للإلقاء والتسليم النهائي

الكلمة الافتتاحية للمناقشة (احفظ هذا الملخص لإلقائه أمام اللجنة):

"بسم الله الرحمن الرحيم، أساتذتي الأفاضل، يسرني تقديم مشروع LOCAL CONTACT، وهو تطبيق اتصال شبكي محلي P2P مستقل تماماً عن السحابة. خلال هذا العمل طورنا النظام على مسارين:

في هندسة البرمجيات: صغنا وثيقة SRS لـ 31 متطلباً وظيفياً، وثبتنا خط الأساس Baseline، وأنشأنا مصفوفة تتبع RTM تربط كل متطلب بالكود والاختبارات. وطبقنا مبادئ SOLID عبر فصل إدارة المفاتيح بواجهة KeyProvider، وحققنا نجاح 33/33 اختباراً آلياً على JVM بنسبة 100%.

في التشفير: حافظنا على AES-256-GCM، واستبدلنا المفتاح الثابت بتبادل مفاتيح ديناميكي ECDH P-256 مع اشتقاق HKDF-SHA256 لتحقيق Forward Secrecy في الدردشة المباشرة. وربطنا هوية الجهاز بمخزن Android Keystore المشفر عتادياً بتوقيع مرتبط بالسياق ونموذج TOFU. وانطلاقاً من الأمانة الأكاديمية، نوضح أن المشروع مجاز ومختبر برمجياً مع APK جاهز، مع توثيق التحسينات المستقبلية مثل حماية Replay الشاملة وفحص الملف الكامل SHA-256. نشكركم ومستعدون لملاحظاتكم"

المراجع والمعايير القياسية المعتمدة (References):

[1] NIST FIPS 197 / SP 800-38D: Advanced Encryption Standard (AES) in Galois/Counter Mode (GCM).
[2] RFC 5869 / RFC 6090: HKDF Key Derivation & Elliptic Curve Cryptography (ECDH secp256r1).
[3] Android Developer Security: Hardware-backed Android Keystore System (TEE / StrongBox).
[4] ISO/IEC/IEEE 29148: Systems and Software Engineering — Requirements Engineering.